プログラミングワークショップ 2

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

目次

- テーマ ブロック崩し2
  - 昨日の整理、追加
  - Gameクラスの修正

今後グロック崩しゲームの開発を実施(3回を予定)

開発したプログラムをKadai_12として提出

機能の整理、追加

- ブロックの数 5 -> 15（横5 縦3）
- メソッドの分割、整理
  - 移動する `void move()`
  - 反射する `void reflect()`
  - 表示する `void display()`
- 画面上部に得点、ボールの数を表示
- 開始画面、結果画面の追加

フィールド追加

追加・修正

```
class Game {

  int life = 3;
  int block_tate  = 3;
  int block_yoko  = 5;
  int blockNum = block_yoko * block_tate;
  PFont font;
  int score = 0;
  int scene = 0;

  Ball ball;
  Bar bar;
  Wall topWall, bottomWall, leftWall, rightWall;
  Block[] blocks = new Block[blockNum];

//以下略
```

コンストラクタ

```
Game() {
    ball = new Ball(width/2, height/2, 0.8, 0.6);
    bar = new Bar(width/2, height*0.8);
    topWall    = new Wall(0, 0, width, 0);
    bottomWall = new Wall(0, height, width, 0);
    leftWall   = new Wall(0, 0, 0, height);
    rightWall  = new Wall(width, 0, 0, height);
    for (int j=0; j<block_tate; j++) {
      for (int i=0; i<block_yoko; i++) {
        blocks[i+block_yoko*j] = new Block(i*80, 80+j*30);
      }
    }
    font = createFont("MS-Gothic", 20);
    textFont(font);
  }

//以下略
```

追加・修正

メソッド追加

```
class Game {
//一部省略

  void move() {
    ball.move();
    bar.move();
  }

  void display() {
    ball.display();
    topWall.display();
    bottomWall.display();
    leftWall.display();
    rightWall.display();
    bar.display();
    for (int i=0; i<blocks.length; i++) {
      blocks[i].display();
    }
    textSize(20);
    textAlign(LEFT);
    text("SCORE " + score, 0, 20);
    text("LIFE " + life, 300, 20);
  }
```

```
 void reflect() {
    if (leftWall.isHitRightSide(ball) ||
      rightWall.isHitLeftSide(ball)) {
      ball.flipXdir();
    }
    if (topWall.isHitBottomSide(ball)) {
      ball.flipYdir();
    }
    if (isDrop()) {
      removeBall();
    }

    if (bar.isHitTopSide(ball) ||
bar.isHitBottomSide(ball)) {
      ball.flipYdir();
    }
    if (bar.isHitRightSide(ball) ||
bar.isHitLeftSide(ball)) {
      ball.flipXdir();
    }
    if (bar.isHitCorner(ball)) {
      ball.flipXdir();
      ball.flipYdir();
    }
```

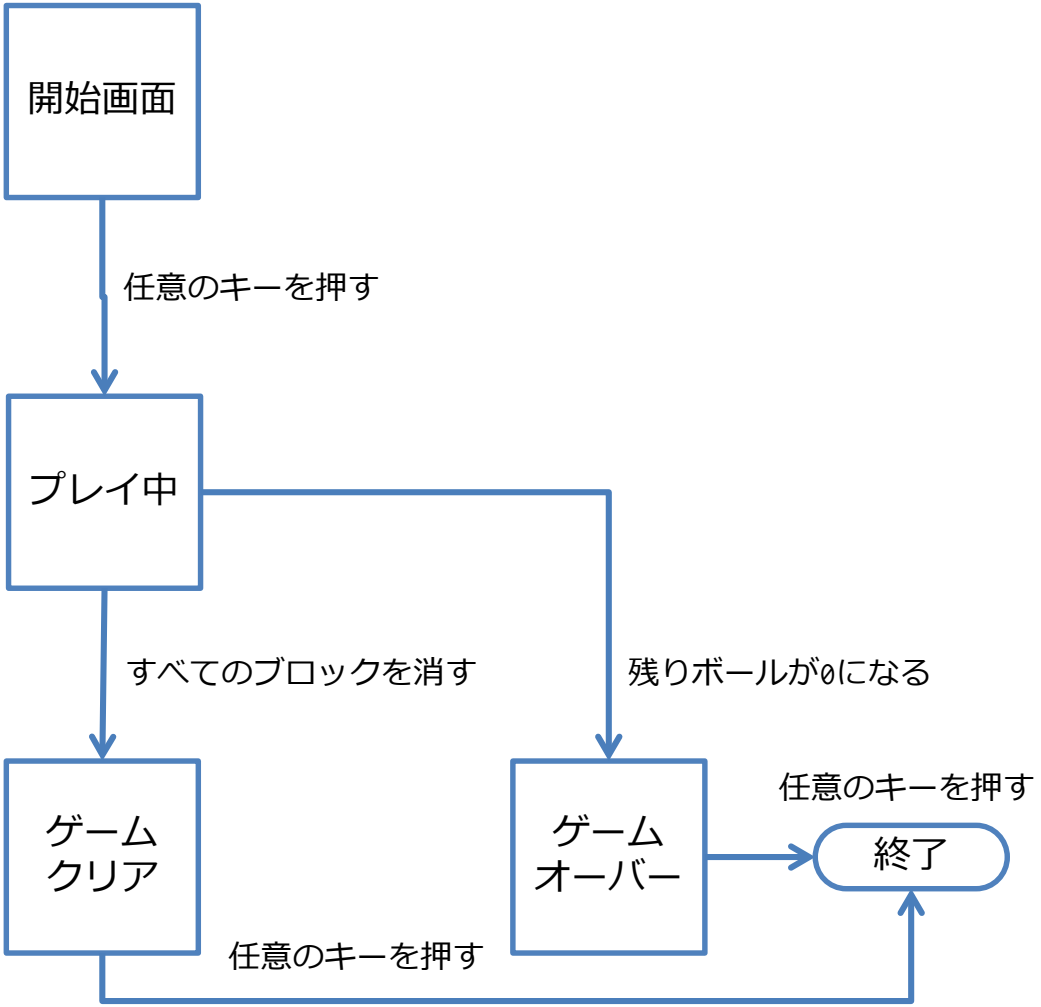
画面遷移

```
    for (int i=0; i<blocks.length; i++) {
      if (blocks[i].isHitTopSide(ball) || blocks[i].isHitBottomSide(ball)) {
        ball.flipYdir();
        blocks[i].remove();
        blockNum--;
        score += blocks[i].score;
      }
      if (blocks[i].isHitRightSide(ball) || blocks[i].isHitLeftSide(ball)) {
        ball.flipXdir();
        blocks[i].remove();
        blockNum--;
        score += blocks[i].score;
      }
      if (blocks[i].isHitCorner(ball)) {
        ball.flipXdir();
        ball.flipYdir();
        blocks[i].remove();
        blockNum--;
        score += blocks[i].score;
      }
    }
  }
//以下略
```

機能の追加

- 画面上部に得点、ボールの数を表示
- 開始画面、結果画面の追加
  - オープニング画面
  - ゲームオーバー時の画面表示
  - ゲームクリア時の画面表示
  - `switch ~ case ~` 文、状態遷移

画面遷移

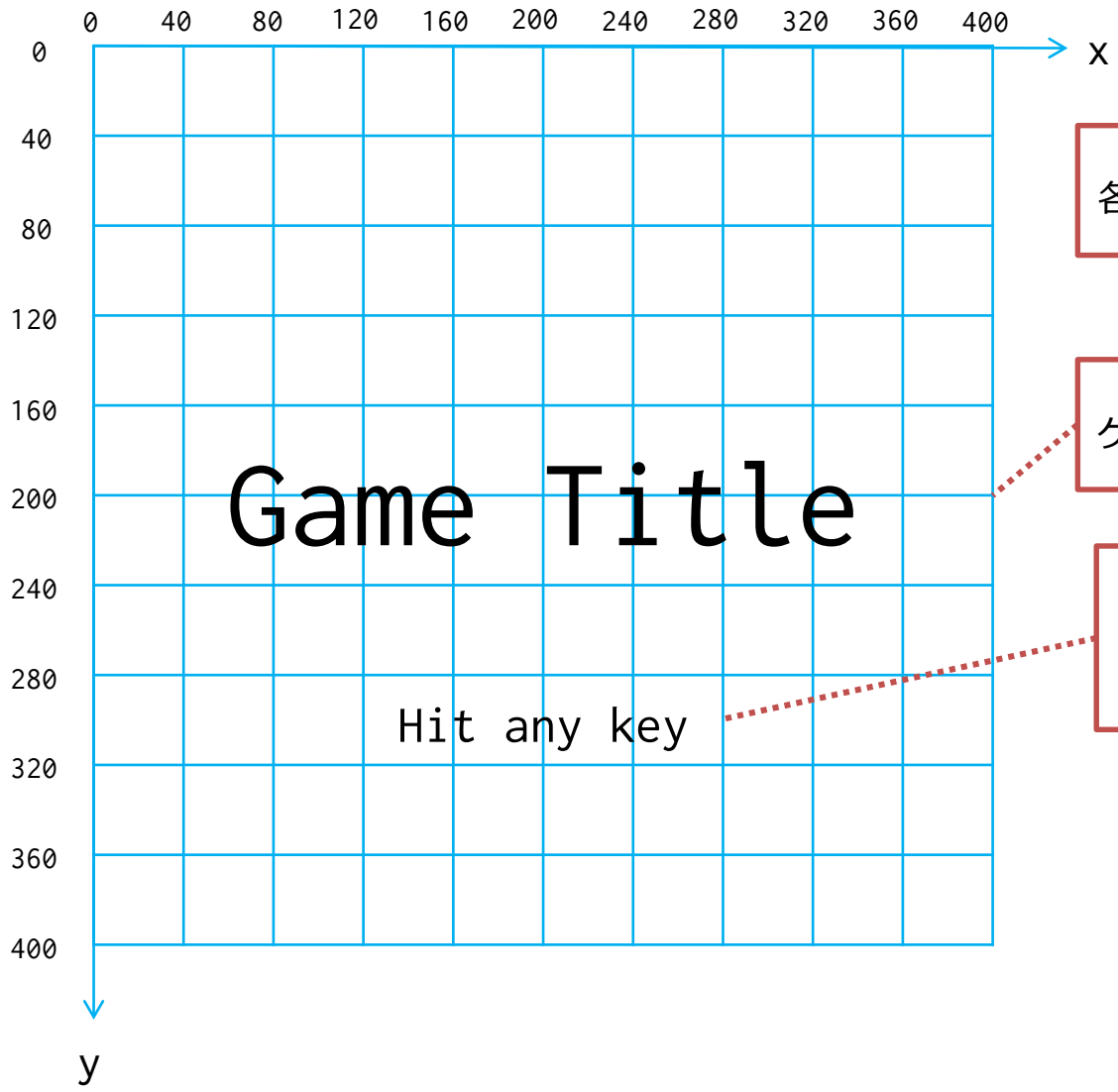

開始画面
任意のキーを押す
プレイ中
すべてのブロックを消す
残りボールが0になる
ゲーム
クリア
ゲーム
オーバー
任意のキーを押す
終了
任意のキーを押す

画面遷移

```
class Game {
//一部省略

void run(){
  background(0);
  switch(scene){
  case 0: //開始画面
    //ここに開始画面の表示と、次の画面に移る処理を書く
    break;
  case 1: //プレイ画面
    //ここにプレイ画面の表示と、次の画面に移る処理を書く
    break;
  case 2: //ゲームオーバー画面
    //ここにゲームオーバー画面の表示と、次の画面に移る処理を書く
    break;
  case 3: //ゲームクリア画面
    //ここにゲームクリア画面の表示と、次の画面に移る処理を書く
    break;
  default: //その他
  break;
  }

  //以下省略
```

開始画面

実行ウィンドウ
400 x 400



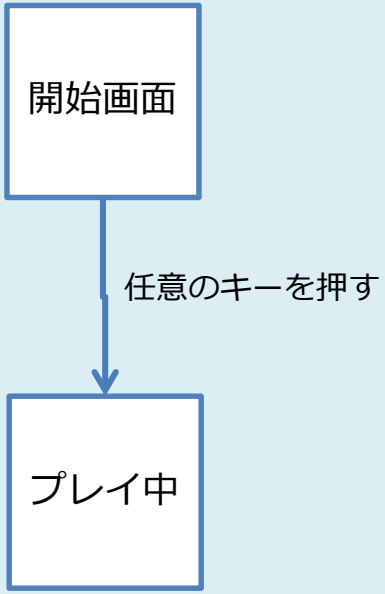
背景は
各自のアイディアで

画面中央に
ゲームタイトル表示

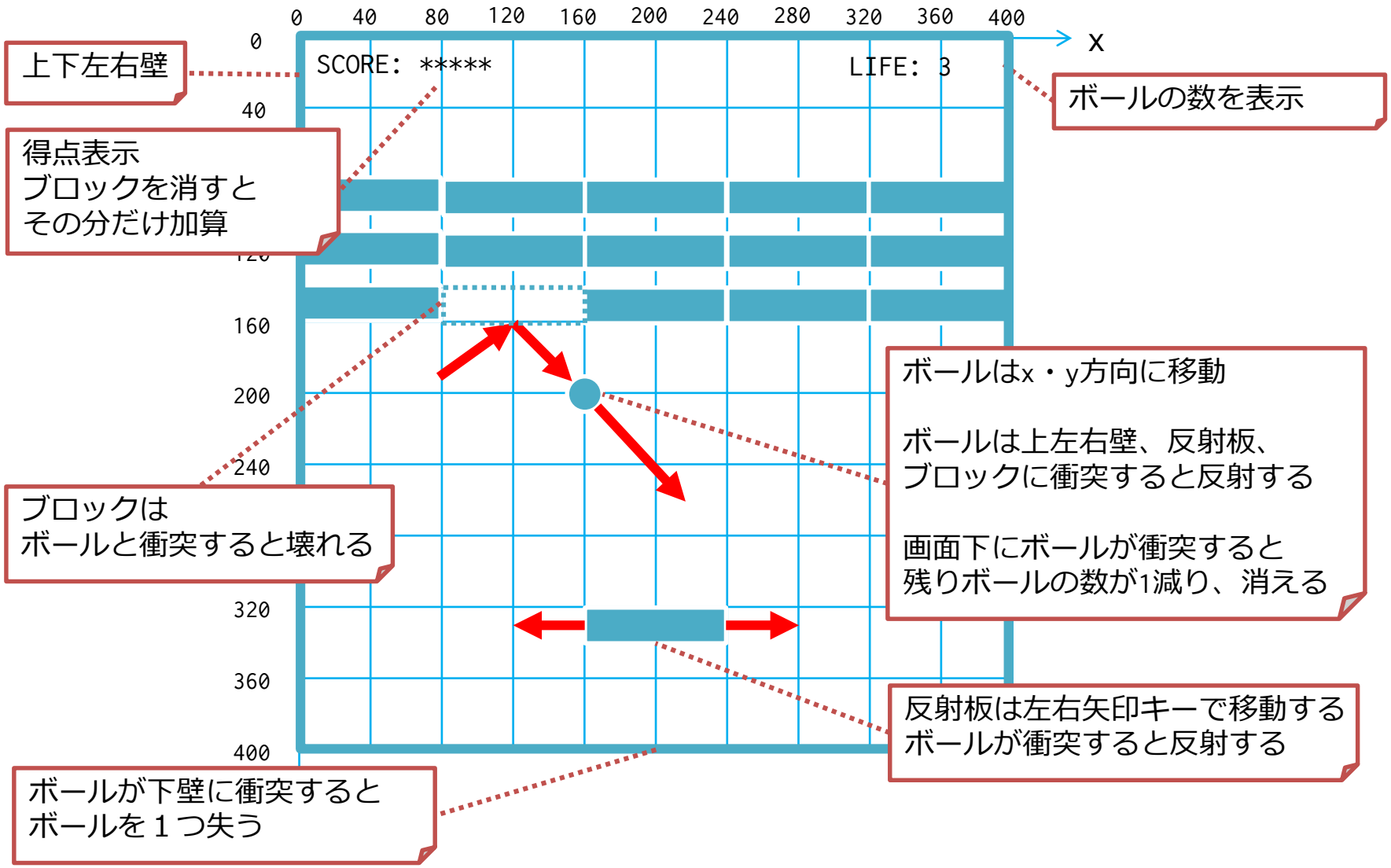
画面下に
次の画面に移る
アクションを表示

開始画面

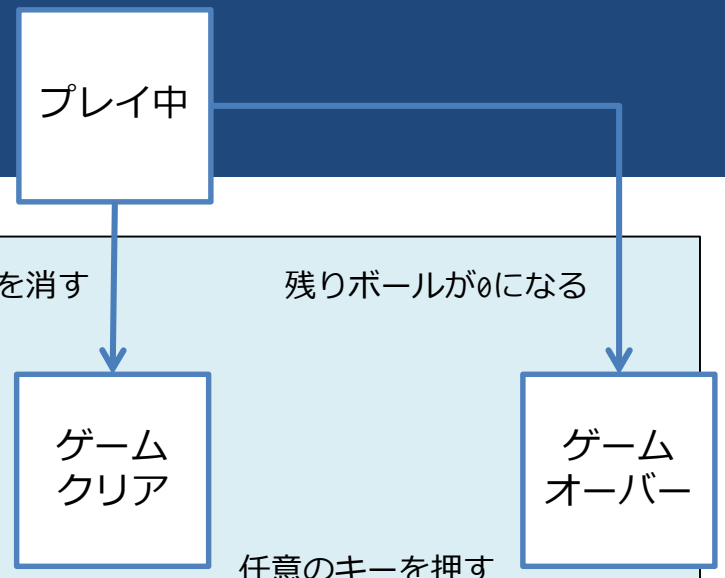
```
//一部省略

  case 0: //開始画面
      textAlign(CENTER);
      textSize(64);
      text("Game Start", width/2, height/2);
      textSize(32);
      text("Hit any key", width/2, 3*height/4);

      if (keyPressed) {          //何かキーを押したら
        scene = 1;               //プレイ画面に遷移
      }
      break;

//以下略
```



プレイ中の画面

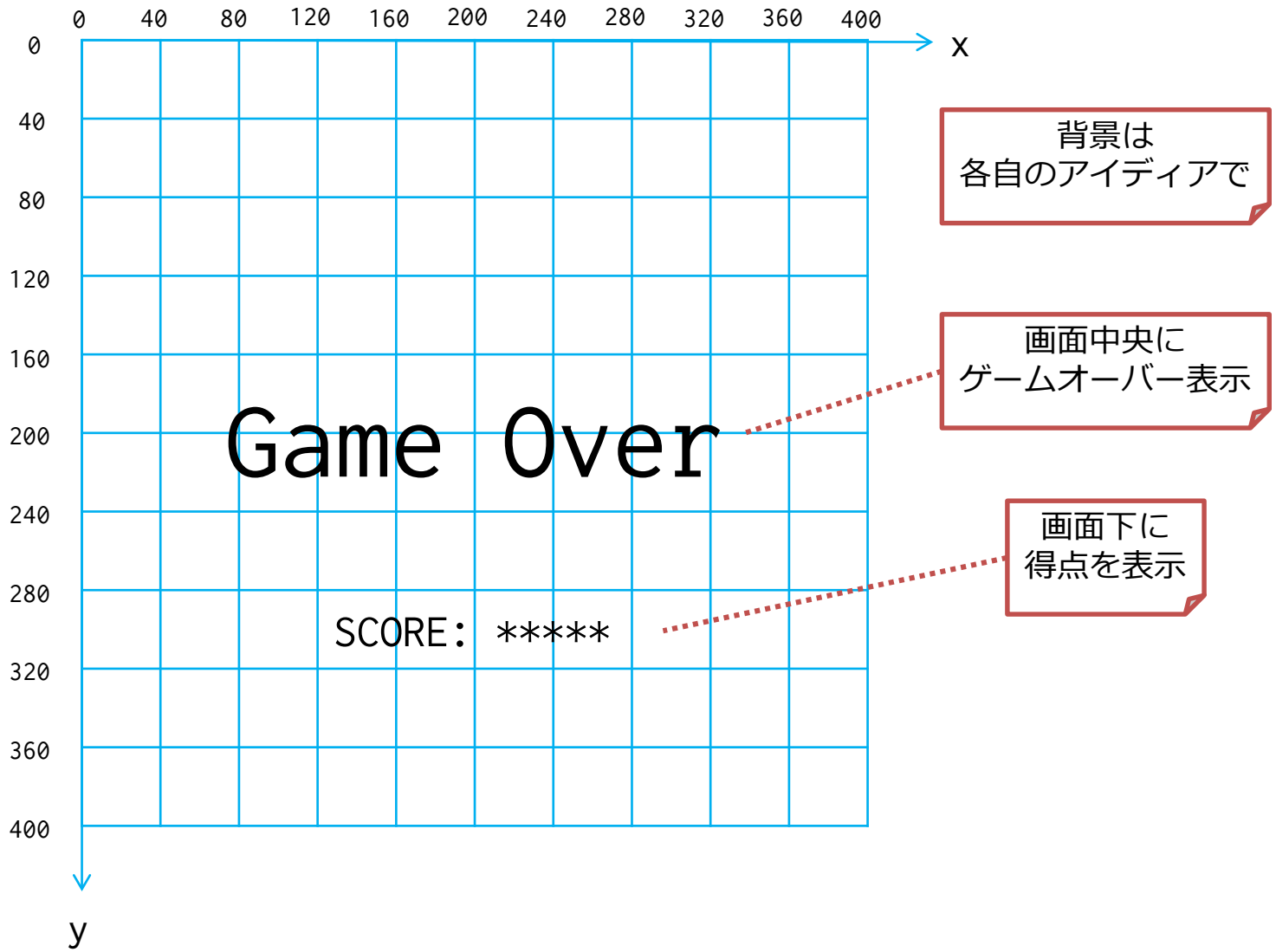

0 40 80 120 160 200 240 280 320 360 400
x
0 40 160 200 240 320 360 400
SCORE: *****
LIFE: 3
上下左右壁
ボールの数を表示
得点表示
ブロックを消すと
その分だけ加算
ボールはx・y方向に移動
ボールは上左右壁、反射板、
ブロックに衝突すると反射する
画面下にボールが衝突すると
残りボールの数が1減り、消える
ブロックは
ボールと衝突すると壊れる
反射板は左右矢印キーで移動する
ボールが衝突すると反射する
ボールが下壁に衝突すると
ボールを１つ失う

プレイ画面



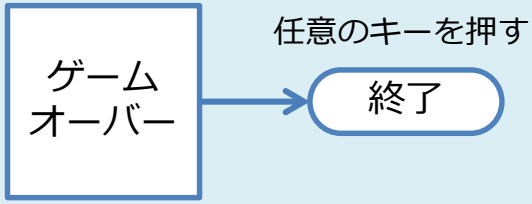

```
//一部省略

 case 1:  //プレイ画面
      if (!isWin() && !isOver()) {
        move();              //移動する
        reflect();           //ボールを反射する
      } else {
        if (isOver()) {      //ゲームオーバーの場合
          state = 2;         //ゲームオーバー画面に遷移
        } else if (isWin()) {//ゲームクリアの場合
          state = 3;         //ゲームクリア画面に遷移
        }
      }
      display();             //表示する
      break;

//以下略
```

ゲームオーバーの画面

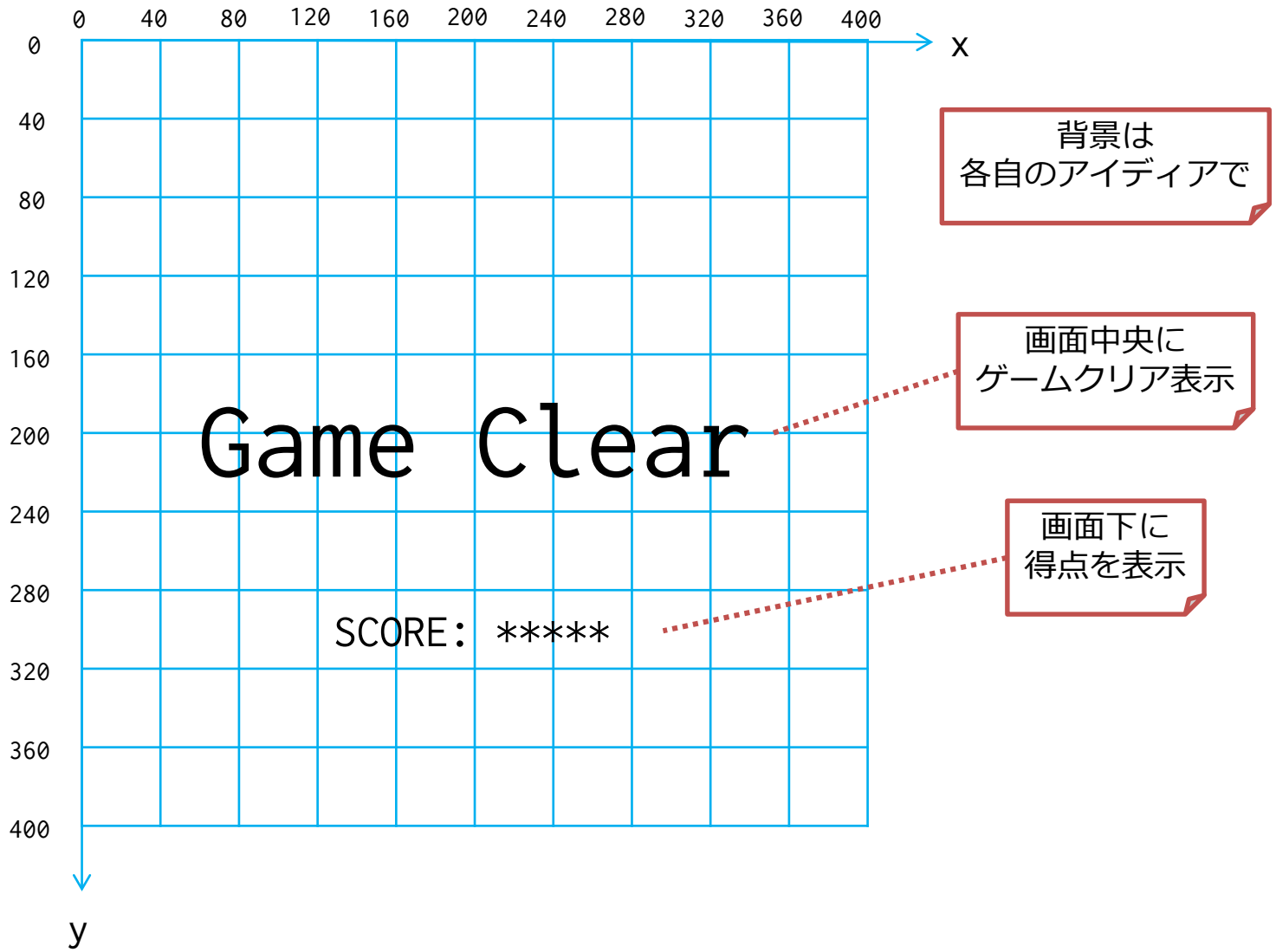

0
40
80
120
160
200
240
280
320
360
400
x
y
Game Over
SCORE: *****
背景は
各自のアイディアで
画面中央に
ゲームオーバー表示
画面下に
得点を表示

ゲームオーバー画面

```
//一部省略

 case 2: //ゲームオーバー
      textSize(64);
      textAlign(CENTER);
      text("GameOver", width/2, height/2);
      textSize(32);
      text("Score : " + score, width/2, 3*height/4);

      if (keyPressed) { //任意のキーを押したら
        exit();         //終了する
      }
      break;

//以下略
```

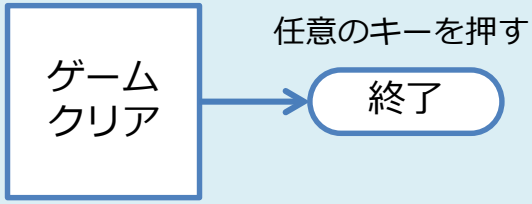


ゲームクリアの画面


0
40
80
120
160
200
240
280
320
360
400
x
y
Game Clear
SCORE: *****
背景は
各自のアイディアで
画面中央に
ゲームクリア表示
画面下に
得点を表示

ゲームクリア画面

```
//一部省略

 case 3: //ゲームクリア
      textSize(64);
      textAlign(CENTER);
      text("Game Clear", width/2, height/2);
      textSize(32);
      text("Score : " + score, width/2, 3*height/4);

      if (keyPressed) { //任意のキーを押したら
        exit();         //終了する
      }

      break;
//以下略
```



Gameクラス①（修正）

```
class Game {

  int life = 3;
  int block_tate  = 3;
  int block_yoko  = 5;
  int blockNum = block_yoko * block_tate;
  PFont font;
  int score = 0;
  int scene = 0;

  Ball ball;
  Bar bar;
  Wall topWall, bottomWall, leftWall, rightWall;
  Block[] blocks = new Block[blockNum];

  Game() {
    ball = new Ball(width/2, height/2, 0.8, 0.6);
    bar = new Bar(width/2, height*0.8);
    topWall    = new Wall(0, 0, width, 0);
    bottomWall = new Wall(0, height, width, 0);
```

Gameクラス②（修正）

```
    leftWall   = new Wall(0, 0, 0, height);
    rightWall  = new Wall(width, 0, 0, height);
    for (int j=0; j<block_tate; j++) {
      for (int i=0; i<block_yoko; i++) {
        blocks[i+block_yoko*j] = new Block(i*80, 80+j*30);
      }
    }
    font = createFont("MS-Gothic", 20);
    textFont(font);
  }

  boolean isWin() {
    if (blockNum <= 0) {
      return true;
    } else {
      return false;
    }
  }
```

Gameクラス③（修正）

```
boolean isOver() {
  if (life <= 0) {
    return true;
  } else {
    return false;
  }
}

boolean isDrop() {
  if (bottomWall.isHitTopSide(ball)) {
    return true;
  } else {
    return false;
  }
}

void removeBall() {
  life--;
  ball = null;
  ball = new Ball(width/2, height/2, 0.5, 0.3);
}
```

Gameクラス④（修正）

```
void run() {
  background(0);

  switch(scene) {
  case 0: //開始画面
    textAlign(CENTER);
    textSize(64);
    text("Game Start", width/2, height/2);
    textSize(32);
    text("Hit any key", width/2, 3*height/4);

    if (keyPressed) { //任意のキーを押したら
      scene = 1;      //プレイ画面に遷移
    }
    break;

  case 1: //プレイ画面
    if (!isWin() && !isOver()) { //ゲームクリアしていない かつ ゲームオーバーしていない
      move();      //移動する
      reflect();   //ボールを反射する
```

Gameクラス⑤（修正）

```
    } else {
      if (isOver()) {        //ゲームオーバーの場合
        scene = 2;           //ゲームオーバー画面に遷移
      } else if (isWin()) {  //ゲームクリアの場合
        scene = 3;           //ゲームクリア画面に遷移
      }
    }
    display();
    break;

  case 2: //ゲームオーバー
    textSize(64);
    textAlign(CENTER);
    text("GameOver", width/2, height/2);
    textSize(32);
    text("Score : " + score, width/2, 3*height/4);
    if (keyPressed) {        //任意のキーを押したら
      exit();                //終了する
    }
    break;
```

Gameクラス⑥（修正）

```
    case 3: //ゲームクリア
      textSize(64);
      textAlign(CENTER);
      text("Game Clear", width/2, height/2);
      textSize(32);
      text("Score : " + score, width/2, 3*height/4);
      if (keyPressed) {        //任意のキーを押したら
        exit();                //終了する
      }

      break;
    default:
      break;
    }
  }

  void move() {
    ball.move();
    bar.move();
  }
```

Gameクラス⑦（修正）

```
void display() {
  ball.display();
  topWall.display();
  bottomWall.display();
  leftWall.display();
  rightWall.display();
  bar.display();
  for (int i=0; i<blocks.length; i++) {
    blocks[i].display();
  }
  textSize(20);
  textAlign(LEFT);
  text("SCORE " + score, 0, 20);
  text("LIFE " + life, 300, 20);
}

void reflect() {
  if (leftWall.isHitRightSide(ball) ||
    rightWall.isHitLeftSide(ball)) {
    ball.flipXdir();
  }
```

追加

Gameクラス⑧（修正）

```
    if (topWall.isHitBottomSide(ball)) {
      ball.flipYdir();
    }
    if (isDrop()) {
      removeBall();
    }

    if (bar.isHitTopSide(ball) || bar.isHitBottomSide(ball)) {
      ball.flipYdir();
    }
    if (bar.isHitRightSide(ball) || bar.isHitLeftSide(ball)) {
      ball.flipXdir();
    }
    if (bar.isHitCorner(ball)) {
      ball.flipXdir();
      ball.flipYdir();
    }

    for (int i=0; i<blocks.length; i++) {
      if (blocks[i].isHitTopSide(ball) || blocks[i].isHitBottomSide(ball)) {
```

Gameクラス⑨（修正）

```
        ball.flipYdir();
        blocks[i].remove();
        blockNum--;
        score += blocks[i].score;
      }
      if (blocks[i].isHitRightSide(ball) || blocks[i].isHitLeftSide(ball)) {
        ball.flipXdir();
        blocks[i].remove();
        blockNum--;
        score += blocks[i].score;
      }
      if (blocks[i].isHitCorner(ball)) {
        ball.flipXdir();
        ball.flipYdir();
        blocks[i].remove();
        blockNum--;
        score += blocks[i].score;
      }
    }
  }
}
```

追加

追加

追加

Blockクラス(修正)

```
class Block extends Rect {

  boolean visible;
  int score = 100;

//以下略
```

追加